

AqKanji2Koe-A Mac マニュアル

株式会社 アクエスト
www.a-quest.com

概要

本文書は、言語処理ライブラリ AqKanji2Koe-A Mac をアプリケーションに組み込んで使用するためのプログラミングの方法、注意点を示したものです。

AqKanji2Koe は、漢字かな混じり文のテキスト情報を AquesTalk 用のアクセント付きの音声記号列に変換する macOS 用のライブラリです。

このライブラリと音声合成ライブラリ AquesTalk を使えば、動的に変化する様々なテキストから、リアルタイムに音声メッセージを生成できるようになります。

特長

- ・簡単に組み込み可能
 - ・ テキスト文字列を入力すると音声記号列を返す、シンプルな API
- ・高速な変換処理
 - ・ 約 20 万字/秒の高速な変換処理
- ・登録単語の追加が可能
 - ・ ユーザ辞書ライブラリを用いて、アプリで単語の追加や、編集、削除が可能
- ・高精度な読み・アクセント付与
 - ・ 最大 50 万語の単語辞書とアクセントルールにより、正確な読みとアクセントを生成

本ライブラリのプログラミングに先立ち動作を確認する場合は、弊社サイトのオンラインデモや、Windows アプリ「AquesTalkPlayer」をお試しください。漢字を含んだ文を音声記号列に変換や、ユーザ辞書を編集する機能を確認できます。

オンラインデモ

<https://www.a-quest.com/demo/>

AquesTalkPlayer

<https://www.a-quest.com/products/aquestalkplayer.html>

本ライブラリを使用するには開発ライセンスキーの設定が必要です。このライセンスキーを設定しない場合は評価版として動作し、以下の制限があります。

評価版の制限

「ナ行、マ行」は、すべて「ヌ」と出力されます

また、本ライブラリをアプリケーションに組み込んで使用する際には**使用ライセンス**、配布には**頒布ライセンス**が必要です。ライセンスの種類や購入方法は、弊社サイトのライセンスのページを参照してください。

仕様

ライブラリ形式	dllib 形式共有ライブラリ Arm64
動作環境	Apple シリコン搭載 Mac (Intel Mac は非対応)
入力データ形式	漢字かな混じり文テキスト(UTF8)
出力データ形式	かな表記音声記号列(UTF8)
関数 I/F	C 関数呼び出し
ビルド環境	Xcode 16.3
ライブラリサイズ	約 150KByte
辞書サイズ	約7MB(約36万語)
依存ライブラリ	/usr/lib/libc++.1.dylib、/usr/lib/libSystem.B.dylib
その他	ユーザ辞書用ライブラリ(AqUsrDic) 同梱

関数 API

libAqKanji2Koe.dylib

AqKanji2Koe_Create

AqKanji2Koe.h

説明	言語処理モジュールのインスタンス生成と内部データの初期化 生成したインスタンスは、使用後、AqKanji2Koe_Release で解放してください。
構文	void * AqKanji2Koe_Create (const char *pathDic, int *pErr)
引数	
pathDic	辞書のディレクトリを指定。通常、"<app dir>/aq_dic"。指定した内容は内部で保存される。 文字列最後の、/ の付与は任意
pErr	エラー時にはエラーコードが入る 正常終了時は不定値
戻り値	インスタンスハンドル エラーの時は0が返る。このとき pErr にエラーコードが設定される。

AqKanji2Koe_Create_Ptr

AqKanji2Koe.h

説明	言語処理モジュールのインスタンス生成と内部データの初期化 生成したインスタンスは、使用後、AqKanji2Koe_Release で解放してください。
構文	void * AqKanji2Koe_Create_Ptr (const void * <i>pSysDic</i> , const void * <i>pUserDic</i> , int * <i>pErr</i>)
引数	
<i>pSysDic</i>	システム辞書(通常 aqdic.bin) をメモリ上に読み込んだ先頭アドレスを指定
<i>pUserDic</i>	ユーザ辞書(通常 aq_user.dic) をメモリ上に読み込んだ先頭アドレスを指定 ユーザ辞書を使用しない場合は NULL を指定する
<i>pErr</i>	エラー時にはエラーコードが入る 正常終了時は不定値
戻り値	インスタンスハンドル エラーの時は0が返る。このとき pErr にエラーコードが設定される。

AqKanji2Koe_Release

AqKanji2Koe.h

説明	言語処理モジュールのインスタンスを開放
構文	void AqKanji2Koe_Release (void * <i>hAqKanji2Koe</i>)
引数	
<i>hAqKanji2Koe</i>	AqKanji2Koe_Create() / AqKanji2Koe_Create_Ptr()で返されたハンドルを指定
戻り値	なし

AqKanji2Koe_Convert

AqKanji2Koe.h

説明	漢字かな混じりのテキストを音声記号列に変換(UTF8 版)
構文	int AqKanji2Koe_Convert (void * <i>hAqKanji2Koe</i> , const char * <i>kanji</i> , char * <i>koe</i> , int <i>nBufKoe</i>)
引数	
<i>hAqKanji2Koe</i>	AqKanji2Koe_Create() / AqKanji2Koe_Create_Ptr()で返されたハンドルを指定
<i>kanji</i>	入力漢字かな混じり文テキスト文字列 (UTF8, NULL 終端)
<i>koe</i>	出力バッファ。音声記号列文字列が返る (UTF8, NULL 終端)
<i>nBufKoe</i>	<i>koe</i> バッファのサイズ[byte] 256 以上を指定。バッファサイズ以上の音声記号列は切り捨てられますので入力テキストの2倍以上のサイズを指定することを推薦。
戻り値	0:正常終了 それ以外:エラーコード

説明	漢字かな混じりのテキストをローマ字音声記号列に変換
構文	int AqKanji2Koe_ConvRoman (void * <i>hAqKanji2Koe</i> , const char * <i>kanji</i> , char * <i>koe</i> , int <i>nBufKoe</i>)
引数	
<i>hAqKanji2Koe</i>	AqKanji2Koe_Create() / AqKanji2Koe_Create_Ptr()で返されたハンドルを指定
<i>kanji</i>	入力漢字かな混じり文テキスト文字列 (Unicode(UTF8), NULL 終端)
<i>koe</i>	出力バッファ。ローマ字音声記号列文字列が返る (ASCII), NULL 終端)
<i>nBufKoe</i>	<i>koe</i> バッファのサイズ[byte] 256 以上を指定。バッファサイズ以上の音声記号列は切り捨てられますので入力テキストの2倍以上のサイズを指定することを推薦。
戻り値	0:正常終了 それ以外:エラーコード

説明	開発ライセンスキーを設定。アプリ起動後、他の関数を呼び出す前に呼び出すことで、以降、製品版とし動作し、評価版の制限がなくなる。
構文	int AqKanji2Koe_SetDevKey (const char * <i>key</i>)
引数	
<i>key</i>	開発ライセンスキー文字列(半角英数)
戻り値	ライセンスキーが正しければ 0、正しくなければ 1 が返る。 不正なキーでも 0 を返す場合がある。このとき制限は解除されない。

libAqUsrDic.dylib (ユーザ辞書ライブラリ)

AqUsrDic ライブラリは、個々の単語の登録を行うインターフェースはありません。アプリケーションでユーザ辞書を変更するときは、CSV 形式の単語リストファイルを介して行います。

説明	CSV 形式の単語リストからユーザ辞書(aq_usr.dic)を生成 生成する aq_usr.dic と同じディレクトリに、システム辞書(aqdic.bin)が必要。
----	--

aq_usr.dic がすでにある場合は上書きする(Append しない)。
【注意】ユーザ辞書は生成時のシステム辞書に依存するため、異なる(サイズの)システム辞書とは使用できない。

構文 int **AqUsrDic_Import**(const char * *pathUserDic*, const char * *pathDicCsv*)

引数

pathUserDic 出力するユーザ辞書(aq_user.dic)ファイルを指定

pathDicCsv CSV 単語リストファイルファイルを指定(入力データ)

戻り値 0:正常終了 それ以外:エラーコード(詳細は AqUsrDic_GetLastError()で取得)

AqUsrDic_Export AqUsrDic.h

説明 ユーザ辞書(aq_usr.dic)から CSV 形式の単語リストを生成
aq_usr.dic と同じディレクトリに、システム辞書(aqdic.bin)が必要。

構文 int **AqUsrDic_Export**(const char * *pathUserDic*, const char * *pathDicCsv*)

引数

pathUserDic ユーザ辞書(aq_user.dic)ファイルを指定

pathDicCsv 出力する CSV 単語リストファイルファイルを指定

戻り値 0:正常終了 それ以外:エラーコード(詳細は AqUsrDic_GetLastError()で取得)

AqUsrDic_Check AqUsrDic.h

説明 単語表記、読みの書式や品詞コードが正しいかチェックする

構文 int **AqUsrDic_Check**(const char * *surface*, const char * *yomi*, int *posCode*)

引数

surface 単語の表記の文字列を指定(UTF8)

yomi 単語の読み(アクセント付き音声記号列)の文字列を指定(UTF8)

posCode 単語の品詞コード(下記参照)を指定

戻り値 0:チェック OK それ以外:NG(詳細は AqUsrDic_GetLastError()で取得)

説明	最後のエラーの詳細メッセージを返す
構文	const char * AqUsrDic_GetLastError()
引数	なし
戻り値	エラーメッセージ (UTF8, NULL 終端)

音声記号列

AqKanji2Koe は、漢字仮名混じりの日本語テキストを入力し、音声記号列を出力します。出力した音声記号列から音声を合成するときは、別途、音声合成ライブラリ **AquesTalk** を用いて音声に変換します。音声記号列の仕様は共通ですので、各種の **AquesTalk** エンジンや各種の OS のライブラリを混在してシステムを構築することができます。

音声記号列の詳細は、付属の音声記号列仕様書を参照してください。

エラーコード表

AqKanji2Koe ライブラリの関数が返すエラーコードの内容は、次の通りです。
AqUsrDic ライブラリに関しては、AqUsrDic_GetLastError() 関数で内容を取得してください。

値	内容
100	その他のエラー
101	関数呼び出し時の引数が NULL になっている。
104	初期化されていない(初期化ルーチンが呼ばれていない)
105	入力テキストが長すぎる
106	システム辞書データが指定されていない
107	変換できない文字コードが含まれている
200 番台	システム辞書(aqdic.bin)が不正
300 番台	ユーザ辞書(aq_user.dic)が不正

CSV 単語リストファイル

ユーザ辞書の編集で使用する CSV 単語リストファイルの例を下に示します。

- 1 列目：単語の表記を記述します。文字コードは UTF8 です。すべて全角で記述します。空白は指定できません。
- 2 列目：単語の読みを音声記号列で指定します(UTF8)。アクセントも ' 記号で指定できます。詳細は音声記号列仕様書の読み記号とアクセント記号の項を参照ください。区切記号は指定できません。
- 3 列目：単語の品詞コード(下記参照)を半角数値で指定します。

CSV 単語リストファイルの例

```
大江戸線,オーエドセン,5
GDGD,グ'ダグダ,1
天王洲,テンノーズ,7
Jアラート,ジェイアラ'ート,5
就活,シューカツ,1
魔理沙,マ'リサ,4
どや顔,ドヤガオ,1
アラフォー,アラフォー,0
```

品詞コード一覧

0	名詞
1	名詞(サ変)
2	人名
3	人名(姓)
4	人名(名)
5	固有名詞
6	固有名詞(組織)
7	固有名詞(地域)
8	固有名詞(国)
9	代名詞
10	代名詞(縮約)
11	名詞(副詞可能)
12	名詞(接続詞的)
13	名詞(形容動詞語幹)
14	名詞(ナイ形容詞語幹)
15	形容詞
16	副詞
17	副詞(助詞類接続)
18	接頭詞(名詞接続)
19	接頭詞(動詞接続)
20	接頭詞(数接続)
21	接頭詞(形容詞接続)
22	接続詞
23	連体詞
24	記号
25	記号(アルファベット)
26	感動詞
27	間投詞

サンプルプログラム

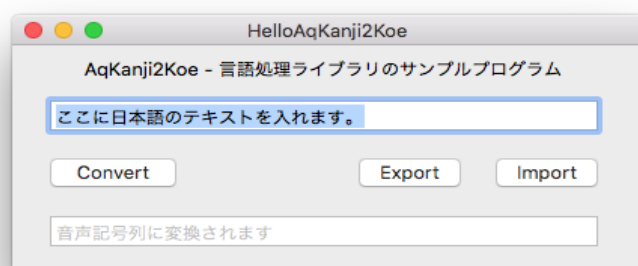
Kanji2KoeCmd と HelloAqKanji2Koe の 2 つのサンプルプログラム (Xcode プロジェクト) が入っています。動作確認環境は Xcode V.16.3、macOS 15.4.1 です。

Kanji2KoeCmd は C++ で記述されたコマンドラインアプリです。漢字仮名テキストを入力し音声記号列を出力します。

```
./Kanji2KoeCmd < sample.txt > sample.koe
```

HelloAqKanji2Koe は、任意の漢字かな混じり文を音声記号列に変換する Swift で記述したアプリケーションです。

ユーザ辞書データを CSV ファイルにエクスポート、逆に CSV ファイルをユーザ辞書データにインポートする機能もあります。



プログラミングガイド

ポイントは、ヘッダファイルのインポートとライブラリの配置・設定です。

ヘッダファイル

ソースコードと同じフォルダにヘッダファイル AqKanji2Koe.h, AqUsrDic.h をコピーします。

C++アプリの場合は、AqKanji2Koe の関数を呼び出すソースコードにインクルードします。

```
#include "AqKanji2Koe.h"
#include "AqUsrDic.h"
```

Swift の場合は、ブリッジングヘッダーを用意し、その中でヘッダファイルをインポートします。関数呼び出し側での #import は不要です。

実際には、アプリ名が HelloAqKanji2Koe の場合は次のファイル名と内容のファイルを作成します。

HelloAqKanji2Koe-Bridging-Header.h

```
#import "AqKanji2Koe.h"
#import "AqUsrDic.h"
```

ブリッジングヘッダーを Xcode ターゲットに指定する方法は次の通りです。

Build Setting タブ → Objective-C Bridging Header :
HelloAqKanji2Koe /HelloAqKanji2Koe-Bridging-Header.h

なお、ユーザ辞書ライブラリ AqUsrDic を使用しない場合は、AqUsrDic の部分は省略できます。

ライブラリファイル

プロジェクトに libAqKanji2Koe.dylib, libAqUsrDic.dylib をコピーします。コピー場所はプロジェクトディレクトリ(.xcodeproj が置かれているフォルダ)がおすすめ。

ビルド時に .dylib を見つけるための設定

Build Settings タブ → Library Search Paths :
ビルド時の libAqKanji2Koe.dylib のパスを指定（例：\$(PROJECT_DIR)）。

Build Phases タブ → Link Binary With Libraries :
libAqKanji2Koe.dylib と libAqUsrDic.dylib を指定。

.dylib は実行時にも必要です。実行時の .dylib の配置場所は各種可能ですが、コマンドラインアプリの場合は実行モジュールと同じフォルダに、.app の場合は Frameworks フォルダ内に配置するのがおすすめです。

```
MyApp.app/
├── Contents/
│   └── Frameworks/
│       ├── libAqKanji2Koe.dylib
│       └── libAqUsrDic.dylib
```

実行時に dylib を見つけるための設定も必要です。

Build Settings タブ → Runpath Search Paths :
実行時の libAqKanji2Koe.dylib のパスを指定します。
実行モジュールと同じフォルダの場合は、
@executable_path
Frameworks フォルダ内の場合は、
@executable_path/../Frameworks

Tips

■実行時に次のようなエラーが出る場合

```
dyld: Library not loaded: libAqKanji2Koe.dylib
```

指定したフォルダに `.dylib` があるか、また、下記の設定をチェックします。

Build Settings → **Runpath Search Paths** に `.dylib` のパスを追加。
設定が正しくても、配置場所によっては **sandbox** で弾かれることがあります。

■ビルド時にアプリバンドル内に `dylib` を自動で入れる設定

Build Phases → **Copy Files** を追加。
Destination を **Frameworks** に設定。
`libAqKanji2Koe.dylib`, `libAqUsrDic.dylib` を追加。

これでビルドされたとき
`MyApp.app/Contents/Frameworks/` に自動で配置されます。

■変数の型

`AqKanji2Koe_Convert()`等の引数や戻り値は C の変数型のため、Swift などを使用する場合は型変換が必要です。これはサンプルプログラムを参考にしてください。

■`AqKanji2Koe_Release()`

`AqKanji2Koe_Create()`で返された `AqKaanji2Koe` インスタンスは、使用後に必ず `AqKanji2Koe_Release()` で解放してください。メモリリークが発生します。

■ユーザ辞書をアプリで動的に変更する

`AqUsrDic` ライブラリは、ユーザ辞書(デフォルト `aq_dic/aq_usr.dic`)を CSV 形式のファイルに変換することと、その逆の CSV 形式のファイルをユーザ辞書に変換する機能しかもっていません。ユーザ辞書に単語を追加するなどの編集機能はアプリ側で用意する必要があります。いったん CSV ファイルに変換した後、CSV ファイルをアプリで編集し、再びユーザ辞書に戻すという流れになります。
なお、ユーザ辞書を変更した場合は、`AqKanji2Koe` も再度 `AqKanji2Koe_Create()` してください。

アプリ開発ガイドライン

アプリケーションの開発(評価での使用を除く)は、以下のガイドラインに従ってください。

ライセンスキー

本ライブラリの動作は、開発ライセンスキーに依存します。このキーは、各ライセンス購入時に発行されるライセンス証に記載されています。

`AqKanji2Koe_SetDevKey ()` をアプリケーションの起動初期（または、`dlopen()`でライブラリをロードした直後）に一度呼び出します。引数には開発ライセンスキーを指定します。これにより製品版として動作し、評価版の制限がなくなります。

※AqKanji2Koe_SetDevKey0の設定は永続化されないので、ライブラリをロードする都度設定する必要があります。

CREDITS

本ライブラリは、BSD ライセンスに基づいてライセンスされている下記のオープンソースソフトウェアを使用しています。このライセンスの表示は SDK 付属の CREDITS ファイルを参照ください。また、本ライブラリを含んだアプリを配布する場合は、CREDITS ファイルまたはその内容が含まれるようにしてください。

- ・ MARISA: Matching Algorithm with Recursively Implemented StorAge
- ・ NAIST Japanese Dictionary：形態素解析用辞書

文書履歴

日付	版	更新内容
2011/01/17	1.0	新規作成
2013/07/02	2.0	Ver.2 用に書き換え
2017/11/25	3.0	Ver.3 用に書き換え
2025/04/26	4.1	Ver.4 用に全面改訂